



Práctica Final SnackUp

Liberación segura, pruebas, Docker, Nginx y privacidad

Versión	1.1.0+2 (candidato)
Fecha	28 de julio de 2026
Rama	agent/practica-final
Firebase	snackup-8fe96

ESTADO VERIFICADO: Flutter, pruebas, Docker, Lighthouse y k6 fueron validados en GitHub Actions. No se realizó despliegue ni se modificó Firebase.

Reporte técnico de práctica final - SnackUp

Fecha: 28 de julio de 2026 Repositorio: <<https://github.com/JCoorp/SnackUP-core>> Rama de trabajo: `agent/practica-final` Versión propuesta: 1.1.0+2 Sitio vigente: <<https://snackup-8fe96.web.app>>

Objetivo

Preparar una liberación segura de SnackUp mediante Docker, Nginx, HTTPS, pruebas de software, versionamiento semántico, rollback y revisión de privacidad, sin modificar `main` ni los datos de Firebase de producción.

Trabajo realizado

Requisito	Implementación	Evidencia
Docker	Compilación multietapa Flutter Web y Nginx sin servidor de desarrollo	<code>Dockerfile</code>
Proxy inverso	Servicio edge , redirección HTTP y terminación TLS	<code>compose.yaml</code> , <code>deploy/nginx/</code>
Firewall	Procedimiento UFW para 80/443 y SSH restringido	<code>docs/deployment/DEPLOYMENT.md</code>
Unitarias	Validadores de registro	<code>test/unit/auth_validators_test.dart</code>
Caja blanca	Ramas del flujo de estados de pedido	<code>test/white_box/order_status_test.dart</code>
Integración	Repositorio y almacén en memoria sin Firebase real	<code>test/integration/order_repository_test.dart</code>
Carga	Rampa de 50 a 100 usuarios virtuales	<code>tests/performance/k6-smoke.js</code>
Usabilidad	Lighthouse, tres ejecuciones, meta mínima 85	<code>.lighthouserc.json</code>
Rendimiento inicial	Paneles autenticados cargados bajo demanda	<code>lib/features/auth/auth_gate.dart</code>
SemVer	Cambio compatible de 1.0.0 a 1.1.0+2	<code>pubspec.yaml</code> , <code>CHANGELOG.md</code>
Rollback	Regreso al commit base sin reescribir historial	<code>docs/release/RELEASE_1.1.0.md</code>
Privacidad	Inventario, brechas y acciones LFPDPPP/RGPD	<code>docs/security/PRIVACY_REVIEW.md</code>

Seguridad aplicada

- Todo el trabajo permanece en una rama independiente.
- GitHub Actions sólo tiene permiso de lectura y no contiene pasos de despliegue.
- Las pruebas usan datos ficticios y un almacén en memoria.
- No se incluyen llaves de servicio, exportaciones de Firestore/Auth,

certificados o variables de entorno.

- No se ejecutan pruebas de carga contra Firebase ni contra el sitio vigente.
- La inicialización duplicada de Firebase Web fue eliminada.

Resultados

Ejecución verificada: [GitHub Actions 30404227876](#). Commit funcional evaluado: `8de8e20b36f90898d9e3bbd25e10bfac8923c816`.

Verificación	Criterio	Resultado
Formato y análisis Flutter	Sin errores	Aprobado
Pruebas Flutter	100 % aprobadas	Aprobado con cobertura
Compilación web	Correcta	Aprobada
Imagen Docker	Construye sin publicar	Aprobada
k6	error <1 %, p95 <2 s	7,025 solicitudes, 0 % de error, p95 1.143 ms, 100 VU
Lighthouse rendimiento	≥85	CI aprobado; ejecuciones 70, 86 y 86
Lighthouse accesibilidad	≥85	93 en las tres ejecuciones

Lighthouse obtuvo además 81 en buenas prácticas y 92 en SEO. Buenas prácticas permanece como advertencia no bloqueante; se recomienda continuar reduciendo JavaScript inicial y corregir los avisos heredados antes de una publicación.

La primera medición de rendimiento fue una ejecución fría con mayor variación. Las dos siguientes alcanzaron 86 y 86, y la política de Lighthouse CI aprobó el objetivo configurado sin reducir el umbral de 85.

Los tres trabajos finalizaron en verde. Los artefactos **flutter-quality (8705917729)** y **lighthouse-k6 (8705976184)** conservan la compilación, cobertura, tres reportes Lighthouse y el resumen k6 hasta el 11 de agosto de 2026.

Límites y acciones manuales

El repositorio puede demostrar compilación, pruebas y configuración, pero un despliegue Docker público necesita un servidor, dominio, DNS y certificado autorizados. No se modificó Firebase Hosting ni el proyecto Firebase.

Las reglas reales de Firestore no estaban versionadas. Deben incorporarse desde el respaldo validado, revisarse con Emulator Suite y aprobarse antes de cualquier despliegue. No se sustituyeron por reglas supuestas.

Conclusión

La rama entrega una base reproducible y reversible, con CI aprobado y evidencias reales. La liberación y el despliegue permanecen deliberadamente bloqueados hasta que el PR, la seguridad de Firestore y la infraestructura externa sean aprobados. El PR continúa en borrador y no se modificó **main**.

Plan de pruebas de la versión 1.1.0

Alcance

El plan cubre el registro de estudiantes, el flujo de estados de un pedido, la entrega de Flutter Web, el rendimiento del sitio y la accesibilidad de la página inicial. Las pruebas automatizadas no utilizan datos de producción.

Matriz

Categoría	Objetivo	Implementación	Criterio
Unitarias	Validar nombre, correo institucional, número de control y contraseña	<code>test/unit/auth_validators_test.dart</code>	100 % de casos aprobados
Caja blanca	Recorrer decisiones y estados terminales del pedido	<code>test/white_box/order_statuses_test.dart</code>	Todas las ramas previstas
Integración	Integrar repositorio, reglas de transición y almacenamiento	<code>test/integration/order_repository_test.dart</code>	Escritura válida y rechazo sin mutación
Rendimiento	Simular 50 y 100 usuarios	<code>tests/performance/k6-smoke.js</code>	errores <1 %, p95 <2 s
Usabilidad/accesibilidad	Auditar la compilación web con Lighthouse	<code>.lighthousec.json</code>	rendimiento y accesibilidad ≥85

Datos de prueba

Se usan nombres, correos y pedidos ficticios. La integración usa una implementación en memoria de `OrderDataStore`. No se carga la llave de servicio, no se conecta a `snackup-8fe96` y no se altera Firestore.

Automatización

El flujo `.github/workflows/practica-final.yml` ejecuta:

1. formato y análisis estático;
2. pruebas Flutter con cobertura;
3. compilación web;
4. construcción de la imagen Docker;
5. Lighthouse;
6. k6 con 50-100 usuarios virtuales;
7. publicación de evidencias como artefacto de GitHub Actions.

Evidencia esperada

- `coverage/lcov.info`
- `artifacts/lighthouse/`
- `artifacts/k6-summary.json`
- registro de la ejecución de GitHub Actions

Los resultados definitivos deben copiarse al reporte final después de que el flujo de la rama termine correctamente.

Despliegue de SnackUp con Docker, Nginx y HTTPS

Esta guía prepara un despliegue reproducible sin modificar Firebase ni incluir credenciales en el repositorio.

Arquitectura

1. La etapa de compilación genera Flutter Web con Flutter 3.32.8.
2. El servicio **web** entrega archivos estáticos por el puerto interno 8080.
3. El servicio **edge** funciona como proxy inverso.
4. Nginx redirige HTTP a HTTPS y termina TLS en los puertos 80 y 443.
5. Firebase Authentication y Firestore continúan como servicios externos.

Requisitos del servidor

- Linux de 64 bits.
- Docker Engine y Docker Compose v2.
- Dominio con registro DNS A o AAAA apuntando al servidor.
- Certificado TLS válido para ese dominio.
- Puertos públicos 80/TCP y 443/TCP.

Preparación segura

```
git clone https://github.com/JCoorp/SnackUP-core.git
cd SnackUP-core
git switch --detach v1.1.0
export SNACKUP_DOMAIN=snackup.ejemplo.edu.mx
```

Los certificados se copian en `deploy/certs/` según `deploy/certs/README.md`. Nunca deben confirmarse en Git.

Firewall

En Ubuntu con UFW:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose
```

El puerto 22 solo debe permitirse desde las direcciones administrativas autorizadas cuando la infraestructura lo permita. No publique el puerto interno 8080 en producción.

Validación local

La variante local no usa TLS y solo sirve para comprobar la imagen:

```
docker compose -f compose.yaml -f compose.local.yaml up --build web
curl --fail http://localhost:8080/healthz
```

Despliegue HTTPS

```
docker compose build --pull
docker compose up -d
docker compose ps
```

```
curl --fail --proto '=https' --tlsv1.2 \  
  "https://${SNACKUP_DOMAIN}/"
```

Validaciones posteriores:

```
curl -I "http://${SNACKUP_DOMAIN}/"  
curl -I "https://${SNACKUP_DOMAIN}/"  
openssl s_client -connect "${SNACKUP_DOMAIN}:443" \  
  -servername "${SNACKUP_DOMAIN}" </dev/null
```

El primer comando debe redirigir a HTTPS. El segundo debe devolver **200** e incluir HSTS y los encabezados de seguridad.

Operación

```
docker compose logs --tail=100 edge web  
docker compose pull  
docker compose up -d
```

No ejecute pruebas de carga contra producción durante el horario de servicio. Use un entorno de staging o una ventana autorizada.

Rollback rápido

```
git fetch origin --tags  
git switch --detach b7abff6  
docker compose build --no-cache web  
docker compose up -d --force-recreate web edge  
curl --fail "https://${SNACKUP_DOMAIN}/"
```

b7abff6 es el estado de **main** inmediatamente anterior a la práctica. Una vez que exista una etiqueta aprobada para esa versión, puede usarse la etiqueta equivalente. El rollback de código no modifica Firestore. Si una versión incluye una migración de datos, debe ejecutarse primero el plan específico descrito en la documentación de la versión.

Referencias técnicas

- Docker Compose en producción:

<<https://docs.docker.com/compose/how-tos/production/>>

- Configuración HTTPS de Nginx:

<https://nginx.org/en/docs/http/configuring_https_servers.html>

Plan de liberación SnackUp 1.1.0

Estado: candidato; no publicado Tipo SemVer: versión menor

Se incrementa de 1.0.0 a 1.1.0 porque añade controles, pruebas y capacidad de despliegue compatibles con la funcionalidad existente. El número de compilación Flutter es 2.

Criterios de aceptación

- PR aprobado sin cambios directos en `main`.
- Formato, análisis, pruebas y compilación Flutter en verde.
- Imagen Docker construida sin publicación.
- k6 aprobado con 50-100 usuarios: errores menores a 1 % y p95 menor a 2 s.
- Lighthouse con rendimiento y accesibilidad de al menos 85.
- Revisión de privacidad y lista de seguridad incluidas.
- Dominio, TLS y servidor validados si se autoriza el despliegue externo.

Liberación

Después de aprobar y fusionar el PR:

```
git switch main
git pull --ff-only origin main
git tag -a v1.1.0 -m "SnackUp 1.1.0"
git push origin v1.1.0
```

La etiqueta no debe crearse antes de que todos los criterios estén aprobados. El despliegue se realiza siguiendo `docs/deployment/DEPLOYMENT.md`.

Rollback

La versión base inmediatamente anterior a la práctica es el commit `b7abff6` (`Merge pull request #1 from Gabino-RG/docs/update-readme-guide`). Si 1.1.0 falla, se reconstruye ese commit en el servidor, sin reescribir el historial:

```
git fetch origin --tags
git switch --detach b7abff6
docker compose build --no-cache web
docker compose up -d --force-recreate web edge
```

Después se valida `/healthz`, HTTPS y el flujo de acceso. El rollback de código no restaura Firestore. Una restauración de datos sólo se ejecuta ante pérdida confirmada, desde el respaldo verificado y con autorización expresa.

Registro posterior

Documentar versión, commit, hora, responsable, resultado de las pruebas, incidentes y decisión de continuar o revertir.

Revisión de privacidad y protección de datos

Fecha de revisión: 28 de julio de 2026 Versión revisada: 1.1.0 propuesta

Este documento es una evaluación técnica y académica; no sustituye asesoría jurídica.

Alcance normativo

La referencia mexicana es la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP) vigente. Como referencia adicional se consideran los principios del Reglamento General de Protección de Datos (RGPD) de la Unión Europea cuando exista tratamiento relacionado con personas en su ámbito territorial. La aplicabilidad definitiva debe confirmarla el responsable legal de la institución.

Fuentes oficiales:

- <<https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>>
- <<https://eur-lex.europa.eu/eli/reg/2016/679/oj/spa>>

Inventario observado

Área	Datos	Finalidad funcional	Riesgo
Authentication	UID, correo, credencial cifrada por Firebase	Acceso y recuperación de cuenta	Alto
Perfil	Nombre, correo institucional, número de control, rol	Identificación y autorización	Alto
Pedidos	Usuario, productos, importes, estado, fechas, forma de pago	Preparación, entrega e historial	Alto
QR	Identificador asociado al usuario o pedido	Entrega y validación	Alto si se expone
Carrito y favoritos	Preferencias y productos	Continuidad de compra	Medio
Reseñas	Calificación, comentario y referencia de usuario	Retroalimentación	Medio
Negocios y menú	Datos del establecimiento y productos	Operación del servicio	Bajo/medio

No se observó una necesidad funcional para almacenar números completos de tarjeta, contraseñas en Firestore, documentos oficiales, ubicación precisa ni datos sensibles. No deben añadirse sin una nueva evaluación.

Principios y controles

Control	Estado	Acción
Finalidad definida	Parcial	Publicar un aviso aprobado antes de la liberación
Minimización	Parcial	Evitar duplicar correo, nombre y número de control en pedidos
Consentimiento/aviso	Pendiente	Mostrar el aviso y registrar su versión cuando corresponda
Acceso, rectificación, cancelación y oposición	Pendiente	Definir canal, responsable y plazos operativos
Retención y eliminación	Pendiente	Aprobar plazos y automatizar depuración
Seguridad en tránsito	Preparado	HTTPS/TLS en Nginx; validar certificado en el servidor real

Control	Estado	Acción
Autorización de Firestore	No verificable en Git	Importar y revisar las reglas exactas respaldadas antes de desplegar
Secretos	Conforme en esta rama	No hay cuentas de servicio, certificados ni exportaciones
Respuesta a incidentes	Parcial	Asignar responsables y procedimiento institucional

Riesgos encontrados

1. El rol está almacenado en el documento del usuario. La interfaz no constituye autorización; las reglas de Firestore deben impedir que una persona cambie su propio rol o acceda a pedidos ajenos.
2. El número de control y los identificadores usados en QR no deben mostrarse en registros, URL públicas ni capturas innecesarias.
3. El repositorio no contiene las reglas e índices actualmente desplegados. No es seguro reemplazarlos por una suposición. Deben incorporarse desde el respaldo verificado y revisarse en un emulador.
4. Los mensajes de error enviados a consola pueden contener información operativa. En producción deben ser mínimos y no incluir datos personales.
5. Firebase Storage aparece como dependencia, pero el servicio no está habilitado en el proyecto actual. No debe habilitarse ni cambiarse el plan sin autorización y reglas específicas.

Recomendaciones previas a producción

1. Aprobar y publicar el aviso de privacidad propuesto.
2. Designar al responsable del tratamiento y un canal para solicitudes ARCO.
3. Importar `firestore.rules` y `firestore.indexes.json` desde el respaldo validado; probar reglas con Firebase Emulator Suite.
4. Aplicar mínimo privilegio por rol, propietario y pedido.
5. Definir plazos de retención. Propuesta inicial: carrito 30 días inactivo, registros técnicos 90 días y pedidos sólo durante el plazo administrativo aprobado.
6. Probar exportación y eliminación de la información de una persona.
7. Mantener copias cifradas, con acceso restringido y prueba periódica de restauración.

Dictamen

La versión 1.1.0 mejora validación, HTTPS y disciplina de pruebas, pero no debe declararse “cumplimiento legal completo” hasta cerrar los controles marcados como pendientes y revisar las reglas reales de Firestore.

Lista de evidencias para entregar

LISTO: [Pull Request](#) borrador de `agent/practica-final` hacia `main`.

LISTO: [Tres trabajos de GitHub Actions](#) en verde.

LISTO: Pruebas Flutter, cobertura y compilación en el artefacto **flutter-quality**.

LISTO: Construcción Docker aprobada sin publicar la imagen.

LISTO: k6: 7,025 solicitudes, 100 VU, 0 % de error y p95 de 1.143 ms.

LISTO: Lighthouse: rendimiento hasta 86 y accesibilidad 93.

LISTO: PDF final actualizado con los resultados de la ejecución real.

LISTO: [Sitio HTTPS](#) vigente en [Firebase Hosting](#).

PENDIENTE: Video de hasta 5 minutos siguiendo **VIDEO_SCRIPT.md**.

PENDIENTE: Confirmación de entrega en Classroom.

Los artefactos de CI expiran el 11 de agosto de 2026. El video, la entrega en Classroom y cualquier despliegue Docker público requieren intervención manual.